

## WIA1002 DATA STRUCTURE GROUP PROJECT

### Project Title: Campus Food Delivery Management System

#### 1. Introduction

This project simulates a simple campus food delivery platform. Students place food orders from different stalls, and riders deliver the orders to hostels or faculty buildings.

The system's main purpose is to manage orders, riders, and basic routing on campus using multiple data structures, to demonstrate a deeper understanding of data structures concepts and use appropriate data structures in application design and development (according to WIA1002 Proforma).

#### 2. Objectives

1. To use multiple data structures (including a graph and at least one priority-based structure) in a single integrated system.
2. To implement and demonstrate operations such as insertion, deletion, searching, traversal, and shortest-path finding on a graph.
3. To design a modular console-based program that handles real-world-like data (orders, locations, routes).

#### 3. System Description

The system manages:

- Food Orders placed by students.
- Riders who deliver the food.
- Campus Locations (canteens, hostels, faculties) connected by routes.

##### 3.1 Order Details

Each order must have at least:

- Order ID (unique)

- Student name
- Pickup location (e.g. “Kolej Makanan”, “Cafeteria A”)
- Delivery location (e.g. “Kolej Kediaman A”, “Fakulti Sains”)
- Order time (or simple sequence number)
- Priority (e.g. 1 = Urgent, 2 = Normal)
- Status (Pending, Assigned, Delivered, Cancelled)

### 3.2 Rider Details

Each rider has:

- Rider ID (unique)
- Rider name
- Current location (one of the campus locations)
- Status (Available, Delivering, Offline)

### 3.3 Campus Map (Graph)

- The campus is modeled as a graph:
  - Nodes = locations (canteens, hostels, faculties).
  - Edges = direct routes between locations.
- Each edge has a distance or cost (can be a simple integer, e.g. 1 for all edges or actual distances).
- The system should be able to find a shortest path (least distance or fewest edges) between two locations.

## 4. Functional Requirements

Your program must support the following operations (via a menu):

### 4.1 Manage Locations & Routes (Graph)

- Add a new location (node).
- Add a route between two locations (edge, with distance).
- Display all locations and their connected neighbours (adjacency list view).

### 4.2 Create New Order

- Input all order details.
- Insert into your data structures.
- Default status = Pending.

### 4.3 View Orders

- View all orders (e.g. in order of creation or by Order ID).
- View orders by status (all Pending, all Assigned, etc.).

### 4.4 Assign Order to Rider (Dispatch)

- Choose a **Pending** order.
- Select an **Available** rider.
- Use the **graph** to compute the shortest path:
  - Rider's current location → Pickup location → Delivery location.
  - Display estimated path (sequence of locations) and total distance.
  - Update:
    1. Order status to Assigned / Delivering.
    2. Rider status to Delivering.

#### 4.5 Complete an Order

- Mark an assigned order as Delivered.
- Update rider's current location to the delivery location.
- Set rider status back to Available.

#### 4.6 Cancel/Delete Order

- Find order by Order ID and cancel/delete it.
- Update any related data (e.g. make rider Available again if needed).

#### 4.7 Search Functions

- Search order by Order ID.
- Search orders by student name or status.
- Search rider by Rider ID.

### 5. Data Structures Requirements

You must use at least **three** different data structures in a meaningful way:

#### 1. Graph (Adjacency List)

- Represent campus locations and routes.
- Use a structure such as:
  1. HashMap<String, List<Edge>>
  2. where Edgestores neighbors and distance.

#### 2. PriorityQueue / Heap (for Orders)

- Store **Pending** orders in a priority queue.
- Highest priority (1 = Urgent) should be dispatched first.
- For orders with the same priority, earlier order time comes first.

### 3. **HashMap / Dictionary**

- Fast lookup of orders by Order ID.
- Fast lookup of riders by Rider ID.

### 4. *(Optional but recommended)* **List / LinkedList / ArrayList**

- Maintain a simple list of all orders (for listing and reporting).

## 5.1 Shortest Path

- Implement a shortest-path search on the graph between two locations.
- You may choose one of the following, depending on what is taught:
  1. **Unweighted graph** → use BFS for minimum number of edges.
  2. **Weighted graph** → use Dijkstra's algorithm for minimum distance.

Clearly state which algorithm you used in your report.

Students must be able to explain:

1. How the graph is stored.
2. How the shortest path algorithm works (step-by-step idea).
3. Time complexity at a surface level (e.g. BFS is  $O(V + E)$ ).

## 6. Program Interface

A console-based menu is sufficient, for example:

```
A console-based menu example: |
1. Manage locations and routes
2. Create new order
3. View orders
4. View riders
5. Assign next order to rider
6. Complete an order
7. Cancel/delete order
8. Search (order/rider)
9. Exit refer me to a photo showing this program interface
```

## 7. Optional Enhancements

Not compulsory, but can give extra impression:

- Save/load locations, orders, and riders to/from text files.
- Allow automatic assignment: system automatically picks the best rider based on distance and availability.
- Display statistics (total orders, average distance, number of urgent orders, etc.).
- Basic “undo last operation” using a stack.

## 8. Project Marks / Rubrics (50%)

### 1. Mastery of Data Structure – 20%

- Show structure design and implementation (data, methods, constructors, etc.).

### 2. Fulfilment of Requirements – 25%

- Traversing / searching works: 10%
- Editing operations (add, delete, sort, etc.) work: 10%
- Results are correct: 5%

### 3. Slides + 5-minute Video – 5%

- Video: step-by-step project rundown, highlight requirements met.
- Slides: group details, screenshots of main functions, requirement checklist
- Showcase competency in mastering the data structure concepts in the existing age of Artificial Intelligence tools.

Project deadline - will be announced on Spectrum.